To verify this, we can rerun:

```
$ docker-machine ls
NAME       ACTIVE DRIVER     STATE   URL        SWARM DOCKER      ERRORS
local-vm-1 *      virtualbox Running tcp://19…       v18.09.1
```

You can see there's a star in the "Active" column for our new instance. If we were using our local Docker Engine, this would instead be -.

What does this mean in practice? All our standard Docker commands will now apply and run against our new VirtualBox instance. For example, if we list our images, rather than outputting the various images we know we have on our local Docker installation, we get an empty list:

```
$ docker images
REPOSITORY    TAG    IMAGE ID    CREATED    SIZE
```

That's because, on our new instance, no images have been built or pulled down yet.

This leads to an important question: how do we get images on the new instance? You may already know, but let's answer this by rerunning our very first Docker command in the book, but this time, let's target our new instance:

```
$ docker run ruby:2.6 ruby -e "puts :hello"
Unable to find image 'ruby:2.6' locally
2.6: Pulling from library/ruby
cd8eada9c7bb: Pull complete
c2677faec825: Pull complete
fcce419a96b1: Pull complete
045b51e26e75: Pull complete
3b969ad6f147: Pull complete
f2db762ad32e: Pull complete
708e57760f1b: Pull complete
06478b05a41b: Pull complete
Digest: sha256:ad724f6982b4a7c2d2a8a4ecb67267a1961a518029244ed943e2d448d6fb7
994
Status: Downloaded newer image for ruby:2.6
hello
```

Running this command causes Docker to pull down the ruby:2.6 image on our new instance, so now when we list the images again, we can see it there:

```
$ docker images
REPOSITORY    TAG    IMAGE ID        CREATED        SIZE
ruby          2.6    f28a9e1d0449    6 days ago     868MB
```

Having configured our Docker CLI to point to the new instance, we can run Docker commands locally and have them seamlessly take effect on the remote